

A Comparative Analysis of MongoDB, CouchDB, and Firebase Firestore with a Case Study of an Integrated Note Learning System

Yoke Yu Lam
Faculty of Computing
Universiti Teknologi Malaysia
Johor, Malaysia
lam.yu@graduate.utm.my

Abstract—This paper compares MongoDB, a document-oriented database, with other NoSQL databases, including CouchDB and Firebase Firestore. An Integrated Note Learning System implemented using MongoDB is also examined. The study concludes that MongoDB occupies a central position in the NoSQL ecosystem by offering a balanced combination of flexible schema design, expressive query capabilities and horizontal scalability.

Keywords—NoSQL databases, document-oriented databases, MongoDB, CouchDB, Firestore

I. INTRODUCTION

NoSQL databases have gained popularity in modern application development due to the limitations of relational databases. As data volumes continue to grow, there is an increasing demand for scalable and flexible database solutions. NoSQL databases, also known as “Not Only SQL” databases, are non-relational databases designed to handle large volumes of unstructured, semi-structured or structured data.

There is a growing diversity of NoSQL systems, making it challenging to choose the suitable solution. Different systems offer varying trade-offs in terms of consistency, scalability, query flexibility and data modelling, and it is important to evaluate their structures and capabilities.

This paper aims to compare MongoDB with other document-oriented databases NoSQL databases including CouchDB and Firebase Firestore. The study focuses on key aspects such as data model, query mechanism, consistency, replication, scalability and use cases. Besides, a case study on an Integrated Note Learning System is conducted.

II. COMPARATIVE ANALYSIS OF DOCUMENT-ORIENTED DATABASES

This section compares document-oriented databases, including MongoDB, CouchDB, and Firebase Firestore.

In document-oriented databases, collections and documents are used to store data instead of tables and rows. This allows flexible and dynamic schema where each document can have different fields, unlike relational databases that enforces rigid columns.

A. Overview of Selected Systems

1) *MongoDB*: MongoDB is among the most widely used NoSQL databases. Since its launch in 2009, it is consistently favoured for managing large-scale, unstructured data (Ciumac et al., 2026).

2) *CouchDB*: CouchDB is developed in the Erlang programming language. It is known for its RESTful HTTP API for database operations (Calçada & Bernardino, 2019).

3) *Firebase Firestore*: Firebase is cloud-based NoSQL database developed by Google. It is designed to support modern web and mobile applications.

B. Comparison

1) *Data model and schema design*: MongoDB utilises Binary JSON (BSON) format, which allows nested structures (Calçada & Bernardino, 2019) and uses the Wired Tiger storage engine. It employs schema-free design that allows effective handling of dynamic and evolving data structures. Each document in MongoDB is identified by a unique identifier and contains key-value pairs (Ciumac et al., 2026).

CouchDB implements JSON documents and uses a B-tree storage engine. Each document has a unique identifier, ‘_id’, similar to that of MongoDB. Besides, it also stores the version of the document, ‘_rev’. Every update creates a new version of the document without deleting the old version (Truica et al., 2015).

Firestore also stores data in JSON-like format. Nested objects or known as maps, can also be stored in a document. In each document there can exist subcollection and subcollection can also contain subcollections, enforcing hierarchical data (Sukmana & Rosmansyah, 2021)

2) *Query mechanisms*: MongoDB uses a Rich query Language with JSON-like syntax to queries documents. It supports aggregation pipelines and batch processing through MapReduce. Besides, join operations between collections is enabled by the ‘\$lookup’ operator (Ciumac et al., 2026), that allows complex data retrieval and transformation.

For CouchDB, queries can be executed using Mango selectors for JSON-based filtering and MapReduce views for complex aggregations. Besides, CouchDB has set of ‘design documents’ where the user can define MapReduce queries to the database. The results of the Map-Reduce function are indexed into ‘views’, which are updated incrementally as updates to the database occur. CouchDB also offers all its operations over a RESTful HTTP API for programmatic access (Matthew, 2015).

Firebase, on the other hand uses Structured query API with built-in indexing and real-time data synchronisation (Kesavan et al., 2023). However, it lacks support for complex joins and advanced aggregation.

3) *Consistency and replication*: CAP theorem, also known as Brewer’s Theorem, states that only two of the three properties can be guaranteed at any given time in a distributed system, namely, Consistency, Availability and Partition Tolerance. Consistency in this context means that at any given time, all nodes in the system see the same data. Availability

ensures that the system always responds to requests. Partition Tolerance means that the system continues to function although communication between some nodes is interrupted. However, network failures can occur and it is not predictable nor avoidable (Gorbenko et al., 2020). On the other hand, replication means the same data is copied and kept on multiple connected machines. Given the trade-offs between consistency and availability, replication provides redundancy and increase data availability with the cost for reduced consistency or increased latency.

MongoDB offers configurable consistency levels to adapt different conditions and read/write concerns. Replication in MongoDB is asynchronous. In a replica set, there is one primary node and one or more secondary nodes. The primary node receives all writes and records every change in an operation log (oplog). The secondary nodes are copies of the primary node, which does not accept writes and replicate data by reading the oplog. In case of primary node failure, the secondary node is voted to be the new primary node (MongoDB, 2015). Data propagation occurs in the background, which allows quick response even when replicas are not fully synchronised (Ciurac et al., 2026).

CouchDB has strong local consistency with eventual cross-node consistency. To manage concurrent access to the database within a node, CouchDB implements Multi-Version Concurrency Control (MVCC). Each read request sees the latest snapshot at the start of the request, regardless of any changes that happen during the read. In term of distributed consistency, CouchDB implements eventual consistency. Each node or server sync periodically and the data is copied incrementally. When conflicts happened, a winning version is picked and the losing version would be kept in history, with the document being marked as conflicted and is managed later (Calçada & Bernardino, 2019). Replication in CouchDB is master-master or multi-master replication and its configurable. There are two types of replications, namely transient replication that is temporary and triggered manually, and persistent replication that is stored as a document and is automatically managed. Replication updates can be done through continuous replication or pull/push-based replication.

Firebase provides strong consistency and implements synchronous replication. Data is split into chunks, known as “splits”. Every split is replicated to different servers, with one of the replicas is selected as the leader. The leader-like coordinator is responsible for handling all write operations. When there is a write request, the leader proposes the change to other replicas, a majority of replicas must agree on the change to allow the write operation to be committed and all replicas apply the change. As all replicas are in sync with the latest committed data and user can read from the replica with strong consistency (Firebase, n.d.).

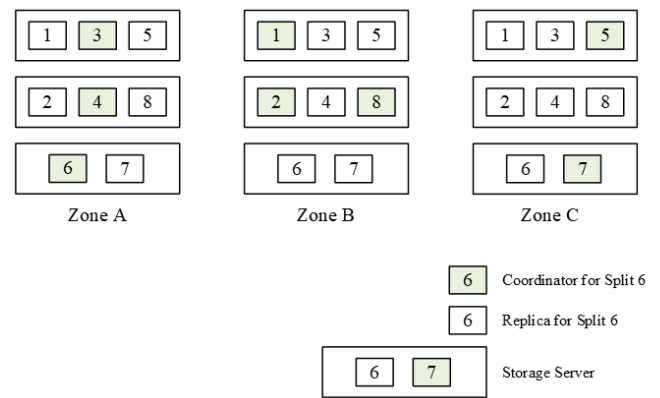


Fig. 1. Cloud Firestore Database Split (Firebase, n.d.).

4) *Scalability*: There are two types of scaling, namely vertical scaling and horizontal scaling. Vertical scaling means to increase the processing power of a single server or cluster. For databases hosted on premise, the hardware of the infrastructure has to be upgraded. For databases on cloud, which can be implemented in MongoDB Atlas and Couchbase Capella, vertical scaling can be upgraded with subscription tier. Firestore on the other hand handles scaling automatically, where it charges per usage. Horizontal scaling brings on additional nodes to share the load which can be done by partitioning. Data is partitioned into data chunks which are distributed across machines and replicated for reliability.

In MongoDB, horizontal partitioning or also known as sharding. Data is split into shards and each shard is implemented as a replica set. Each replica set is then replicated to multiple nodes. In MongoDB, the developers define the shard key to control the partitioning strategy (MongoDB, 2015).

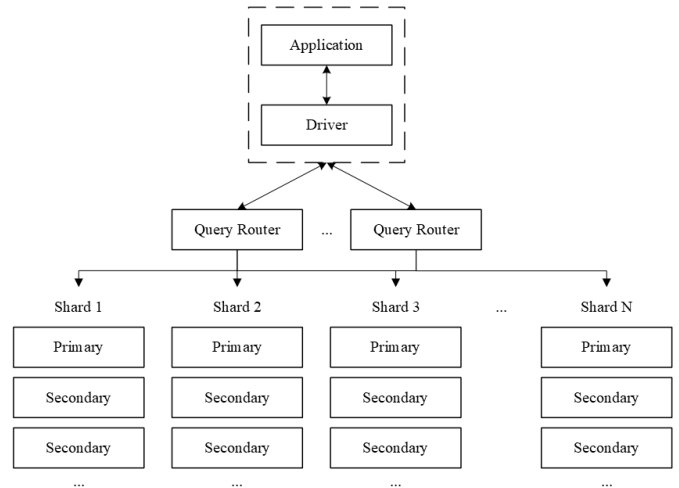


Fig. 2. Architecture of MongoDB in sharding (MongoDB, 2015)

CouchDB also partition data into shards and distribute copies of shards to different nodes in a cluster. In CouchDB, the databases are automatically shard and distributed, but the sharding behaviour can be configured.

Firebase splits data into “splits” and stored across distributed infrastructure. In Firebase, the partitioning is automatic and is handled fully by the system (Firebase, n.d.).

5) *Use case*: MongoDB is typically a general-purpose document database, with high flexibility and high read and write performance. Thus, MongoDB is well-suited for a wide range

of applications, including content management systems, flexible schema application and real-time analytics.

CouchDB on the other hand is optimised for offline-first apps and distributed systems due to its built-in replication mechanism and eventual synchronization.

Firebase is optimised for real-time apps due to its automatic data synchronisation and real-time update propagation. Hence, Firebase is most suited for mobile and web apps, which required real-time sync (Firebase, n.d.).

TABLE I. COMPARISON TABLE FOR MONGODB, COUCHDB AND FIRESTORE

Aspects	Databases		
	<i>MongoDB</i>	<i>CouchDB</i>	<i>Firestore</i>
Data model	BSN	JSON	JSON with hierarchical data
Query Mechanism	Rich query language	Mango queries, MapReduce views, HTTP/REST API access	Structured query API
Consistency	Eventual consistency, configurable	Strong consistency within a node, eventual consistency across nodes	Strong consistency
Replication	Asynchronous replication, Leader-Based Replication	Asynchronous replication, master-master / multi-master replication, configurable	Synchronous replication, Leader-Based Replication
Scalability	Sharding, controlled by user	Sharding, automatic but configurable	Partitioning, fully automatic
Use Cases	General-purpose apps, big data, flexible schema application	Offline-first apps, distributed systems	Mobile/web apps, real-time collaborative applications

C. Discussion

Within document-oriented databases, MongoDB and CouchDB share similarities in schema design and data models. However, MongoDB has a richer query capability, making it more suitable for applications that require complex data retrieval and analytics. While CouchDB emphasises offline-first capabilities through its replication model.

In terms of consistency, MongoDB provides configurable consistency levels, allowing developers to adjust according to different needs. In contrast with the limited consistency configurability in CouchDB and Firestore compared to MongoDB.

In terms of scalability, MongoDB supports horizontal scaling through user-defined sharding. This contrast with Firestore, which is fully managed.

Overall, MongoDB is a general-purpose NoSQL database that balances flexibility, scalability and query capability. While it may not be able to achieve the offline properties in CouchDB or real-time sync in Firestore, provides a versatile solution suitable for many modern applications.

CASE STUDY

An Integrated Note Learning System is built upon MongoDB according to Kate et al. (2026). It serves as a practical example of a data-intensive platform requiring efficient handling of structured and unstructured data. The system integrates notes, quizzes, multimedia content and semantic search.

MongoDB is adopted due to its flexible schema and its ability to manage different data with different characteristics through collections. The Atlas Vector Search enables semantic retrieval using embeddings, which allow users to perform semantic search across the learning materials. This capability is particularly significant for adaptive learning environments where traditional keyword-based retrieval is insufficient.

Compared to CouchDB and Firebase, MongoDB provides a stronger support for complex querying and semantic search integration. While CouchDB focuses more on replication and synchronisation and Firebase emphasises on real-time data, neither provides native support for vector-based retrieval at the same level. This makes MongoDB more suitable for systems requiring advanced data processing and retrieval.

However, there are also challenges in the implementation including the increase of complexity and computational overhead due to the AI-based embedding models.

CONCLUSION

In conclusion, MongoDB occupies a central position within the NoSQL ecosystem, particularly among document-oriented databases. Compared to CouchDB, MongoDB provides more advanced querying and analytics feature. When compared with Firestore, MongoDB offers greater control over deployment and data distribution. Overall, this analysis shows that MongoDB is a strong general-purpose NoSQL database.

REFERENCES

- Calçada, A., & Bernardino, J. (2019). *Evaluation of Couchbase, CouchDB and MongoDB using OSSpal*. 11th International Joint Conference on Knowledge Discovery, Knowledge Engineering and Knowledge Management (IC3K 2019). <https://doi.org/10.5220/0008345104270433>
- Ciumac, M., Györödi, C. A., Györödi, R. S., & Costea, F. M. (2026). Performance evaluation of MongoDB and RavenDB in IIOT-Inspired Data-Intensive mobile and web applications. *Future Internet*, 18(1), 57. <https://doi.org/10.3390/fi18010057>
- Firebase. (n.d.). *Understand reads and writes at scale*. <https://firebase.google.com/docs/firestore/understand-reads-writes-scale>
- Gorbenko, A., Karpenko, A., & Tarasyuk, O. (2020). *Analysis of Trade-offs in Fault-Tolerant Distributed Computing and Replicated Databases* (pp. 1–6). 11th IEEE International Conference on Dependable Systems, Services and Technologies, DESSERT'2020. <https://doi.org/10.1109/dessert50317.2020.9125078>
- Kate, P., Dagadkar, S., & Sontakke, A. (2026). Smart Note Management Learning Hub (pp. 1848–1853). 2026 7th International Conference on Mobile Computing and Sustainable Informatics (ICMCSI). <https://doi.org/10.1109/icmcsi67283.2026.11412869>
- Kesavan, R., Gay, D., Thevessen, D., Shah, J., & Mohan, C. (2023). *Firestore: The NoSQL Serverless Database for the Application Developer* (pp. 3376–3388). International Conference on Data Engineering. <https://doi.org/10.1109/icde55515.2023.00259>
- Matthew, H. (2015). A Case Study for NoSQL Applications and Performance Benefits: CouchDB vs. Postgres.
- MongoDB. (2015). *MongoDB Architecture Guide: MongoDB 3.2* [White paper].
- Truica, C., Radulescu, F., Boicea, A., & Bucur, I. (2015). *Performance Evaluation for CRUD Operations in Asynchronously Replicated Document Oriented Database* (pp. 191–196). 2015 20th International Conference on Control Systems and Computer Science, CSCS 2015. <https://doi.org/10.1109/cscs.2015.32>

Log

13/4/2026 (Sun)	Simple research on the topic
17/4/2026 (Fri)	Paper structure and topic setting AI Prompts keyword: review structure, suggest key aspects, suggest topic, suggest databases Structure: 1. Introduction 2. Background 3. Comparative analysis between document-oriented database a. Overview of selected systems b. Comparison c. Use case d. Discussion 4. Comparative analysis between different NoSQL database types 5. Discussion 6. Conclusion Key Aspects 1. Data model and schema design 2. Query mechanisms 3. Consistency and replication 4. Performance and scalability Databases 1. Document-oriented: MongoDB, CouchDB, Firebase Firestore 2. Cross-types: MongoDB, Cassandra, Redis, Neo4j Simple background 1. Database 2. NoSQL database 3. MongoDB
18/4/2026 (Sat)	Simple research on document-oriented databases
19/4/2026 (Sun)	Simple research on document-oriented databases
20/4/2026 (Mon)	Begin writing on document-oriented databases AI prompts keywords: MongoDB, CouchDB, Firestore Firebase, Comparative Analysis of MongoDB, CouchDB, Firestore Firebase Realised that there are very limited resources form academic paper and conference papers, turned source focus to database official documentation. Wrote part of document-oriented databases
21/4/2026 (Tue)	Wrote part of document-oriented databases Changed structure of comparison to 1. Data model and schema design 2. Query mechanisms 3. Consistency and replication 4. Scalability 5. Use case
24/4/2026 (Thu)	Completed document-oriented databases AI Prompt: check correctness
24/4/2026 (Fri)	Wrote part of cross-type databases AI prompt keywords: Cassandra, Redis, Neo4j Dropped Redis from comparison due to the complexity in its official document website
25/4/2025 (Sat)	Wrote part of cross-type databases

26/4/2025 (Sun)	Completed cross-type databases Completed discussion, conclusion, abstract Combined introduction and background due to content length Realised the content somehow strayed away from the assignment requirements a bit 🤖 Doing some redo Decided to drop the entire cross-type comparison Kept here in case I need it III. COMPARATIVE ANALYSIS OF CROSS-TYPE NOSQL DATABASES In this section, a comparative analysis is conducted among cross-type NoSQL databases, including MongoDB, Apache Cassandra and Neo4j. <i>A. Overview of Selected Systems</i> 1) <i>Apache Cassandra</i>: Cassandra is an open-source, distributed NoSQL database that implements a partitioned wide-column storage model. 2) <i>Neo4j</i>: Neo4j is a native graph database that implements graph model all the way to the storage level (Neo4j, n.d.-c). <i>B. Comparison</i> 1) <i>Data model and schema design</i>: Cassandra implements a partitioned wide-column storage model. Partitions of data contain multiple rows, each of which contains a flexible set of individually typed columns. Each row is uniquely identified by partition key and one or more clustering keys. Joins are not supported in Cassandra so denormalisation is required. In Neo4j, data model describes the nodes and relationships in the domain and includes labels, types and properties. 2) <i>Query mechanism</i>: Cassandra provided the Cassandra Query Language (CQL), a SQL-like language (Apache Cassandra, n.d.-a). Neo4j uses Cypher as its query language. It is similar to SQL, but optimised for graphs (Neo4j, n.d.-c). 3) <i>Consistency and replication</i>: Cassandra implements a tunable consistency model, with eventually consistent semantics by default and implements multi-master replication. Consistency is controlled using consistency level, which determines the number of acknowledgements before confirming read/write success (Gorbenko et al., 2020). Write operation are always sent to all replicas, but the coordinator only waits for the configured number of acknowledgments before confirming success. Cassandra uses a last-write-wins model and every mutation is timestamped and the latest version of data is the “winning” value. Reads may contact multiple replicas and return the latest value based on timestamps. Cassandra has several best-effort techniques to resolve convergence of replicas, including ‘Replica read repair’ and ‘Hinted handoff’. To guarantee eventual consistency, ‘anti-entropy repair’ is implemented (Apache Cassandra, n.d.-b). Neo4j implements causal consistency. When invoked, a client application is guaranteed to read at least its own writes. Replicas in Neo4j are classified to primary and secondary, stored in different servers. Primaries participate in write operations. At any given time, one primary is automatically elected as a writer. The database writer synchronously pushes writes to other primaries and does not allow a commit to be completed until it receives confirmation that the data has been replicated to other primaries. Secondary copies asynchronously replicate from primaries via transaction log shipping. The database secondaries are to scale out read workloads. The number of primary and secondary can be configured depending on the needs. The minimum is single primary (Neo4j, n.d.-a). 4) <i>Scalability</i>: Cassandra achieves horizontal scalability by partitioning data stored in the system using a hash function. Each partition is replicated to multiple physical nodes (Apache Cassandra, n.d.-b). Neo4j provides data federation through composite databases, allowing queries across multiple databases. However, Neo4j does not natively support automatic horizontal partitioning of graph data, as distributing relationships across nodes is complex (Neo4j, n.d.-b). 5) <i>Use case</i>: Apache Cassandra is designed for large-scale distributed environments that require high availability and fault tolerance. Therefore, Cassandra is well-suited for use cases such as big data platforms, time-series data, IoT applications, and systems that require high write throughput and continuous availability. Neo4j is optimised for applications that involve highly connected data and complex relationships. The graph data model enables efficient traversal of relationship between entities. Therefore, Neo4j is well-suited for social networks, recommendation systems, fraud detection and knowledge graphs.
-----------------	--

TABLE II. COMPARISON TABLE FOR MONGODB, CASSANDRA AND NEO4J

Aspects	Databases		
	<i>MongoDB</i>	<i>Cassandra</i>	<i>Neo4j</i>
Data model	Document-based	Wide-column	Graph
Schema Design	Flexible	Semi-structured	Structured relationship
Query Mechanism	Rich query language	Cassandra Query Language (CQL)	Cypher
Consistency	Eventual consistency, configurable	Eventual Consistency, configurable	Causal consistency
Replication	Asynchronous replication, Leader-Based Replication	Multi-master replication	Synchronous among primaries, asynchronous among secondaries, leader-based replication
Scalability	Sharding, controlled by user	Partitioning using consistent hashing	Data federation
Use Cases	General-purpose apps, big data, flexible schema application	Big data, analytics, logging	Social networks, recommendation systems

Apache Cassandra. (n.d.-a). *Dynamo*. Apache Cassandra Documentation. <https://cassandra.apache.org/doc/stable/cassandra/architecture/dynamo.html>

Apache Cassandra. (n.d.-b). *The Cassandra Query Language (CQL)*. Apache Cassandra Documentation. <https://cassandra.apache.org/doc/latest/cassandra/developing/cql/index.html>

Neo4j. (n.d.-a). *Introduction: Neo4j clustering architecture - Operations Manual*. Neo4j Graph Data Platform. <https://neo4j.com/docs/operations-manual/current/clustering/introduction/>

Neo4j. (n.d.-b). *Scaling with Neo4j - Operations Manual*. Neo4j Graph Data Platform. <https://neo4j.com/docs/operations-manual/current/scalability/scaling-with-neo4j/>

Neo4j. (n.d.-c). *What is Neo4j?* Neo4j Graph Data Platform. <https://neo4j.com/docs/getting-started/whats-neo4j/>

27/4/2026 (Mon)

Searching for case study
 Completed Case Study
 Updated structure of paper
 Updated Abstract, Introduction, Discussion, Conclusion
 AI Prompts: proofread and comments

Assignment 1 Lam Yoke Yu.pdf

by Yoke Yu Lam

Submission date: 26-Apr-2026 11:55PM (UTC-0700)

Submission ID: 2945185771

File name: Assignment_1_Lam_Yoke_Yu.pdf (263.07K)

Word count: 2231

Character count: 13473

A Comparative Analysis of MongoDB, CouchDB, and Firebase Firestore with a Case Study of an Integrated Note Learning System

Yoke Yu Lam
Faculty of Computing
Universiti Teknologi Malaysia
Johor, Malaysia
lam.yu@graduate.utm.my

Abstract—This paper compares MongoDB, a document-oriented database, with other NoSQL databases, including CouchDB and Firebase Firestore. An Integrated Note Learning System implemented using MongoDB is also examined. The study concludes that MongoDB occupies a central position in the NoSQL ecosystem by offering a balanced combination of flexible schema design, expressive query capabilities and horizontal scalability.

Keywords—NoSQL databases, document-oriented databases, MongoDB, CouchDB, Firestore

I. INTRODUCTION

NoSQL databases have gained popularity in modern application development due to the limitations of relational databases. As data volumes continue to grow, there is an increasing demand for scalable and flexible database solutions. NoSQL databases, also known as “Not Only SQL” databases, are non-relational databases designed to handle large volumes of unstructured, semi-structured or structured data.

There is a growing diversity of NoSQL systems, making it challenging to choose the suitable solution. Different systems offer varying trade-offs in terms of consistency, scalability, query flexibility and data modelling, and it is important to evaluate their structures and capabilities.

This paper aims to compare MongoDB with other document-oriented databases NoSQL databases including CouchDB and Firebase Firestore. The study focuses on key aspects such as data model, query mechanism, consistency, replication, scalability and use cases. Besides, a case study on an Integrated Note Learning System is conducted.

II. COMPARATIVE ANALYSIS OF DOCUMENT-ORIENTED DATABASES

This section compares document-oriented databases, including MongoDB, CouchDB, and Firebase Firestore.

In document-oriented databases, collections and documents are used to store data instead of tables and rows. This allows flexible and dynamic schema where each document can have different fields, unlike relational databases that enforces rigid columns.

A. Overview of Selected Systems

1) *MongoDB*: MongoDB is among the most widely used NoSQL databases. Since its launch in 2009, it is consistently favoured for managing large-scale, unstructured data (Ciurac et al., 2026).

2) *CouchDB*: CouchDB is developed in the Erlang programming language. It is known for its RESTful HTTP API for database operations (Calçada & Bernardino, 2019).

3) *Firebase Firestore*: Firebase is cloud-based NoSQL database developed by Google. It is designed to support modern web and mobile applications.

B. Comparison

1) *Data model and schema design*: MongoDB utilises Binary JSON (BSON) format, which allows nested structures (Calçada & Bernardino, 2019) and uses the Wired Tiger storage engine. It employs schema-free design that allows effective handling of dynamic and evolving data structures. Each document in MongoDB is identified by a unique identifier and contains key-value pairs (Ciurac et al., 2026).

CouchDB implements JSON documents and uses a B-tree storage engine. Each document has a unique identifier, ‘_id’, similar to that of MongoDB. Besides, it also stores the version of the document, ‘_rev’. Every update creates a new version of the document without deleting the old version (Truica et al., 2015).

Firestore also stores data in JSON-like format. Nested objects or known as maps, can also be stored in a document. In each document there can exist subcollection and subcollection can also contain subcollections, enforcing hierarchical data (Sukmana & Rosmansyah, 2021)

2) *Query mechanisms*: MongoDB uses a Rich query Language with JSON-like syntax to queries documents. It supports aggregation pipelines and batch processing through MapReduce. Besides, join operations between collections is enabled by the ‘\$lookup’ operator (Ciurac et al., 2026), that allows complex data retrieval and transformation.

For CouchDB, queries can be executed using Mango selectors for JSON-based filtering and MapReduce views for complex aggregations. Besides, CouchDB has set of ‘design documents’ where the user can define MapReduce queries to the database. The results of the Map-Reduce function are indexed into ‘views’, which are updated incrementally as updates to the database occur. CouchDB also offers all its operations over a RESTful HTTP API for programmatic access (Matthew, 2015).

Firebase, on the other hand uses Structured query API with built-in indexing and real-time data synchronisation (Kesavan et al., 2023). However, it lacks support for complex joins and advanced aggregation.

3) *Consistency and replication*: CAP theorem, also known as Brewer’s Theorem, states that only two of the three properties can be guaranteed at any given time in a distributed system, namely, Consistency, Availability and Partition Tolerance. Consistency in this context means that at any given time, all nodes in the system see the same data. Availability

ensures that the system always responds to requests. Partition Tolerance means that the system continues to function although communication between some nodes is interrupted. However, network failures can occur and it is not predictable nor avoidable (Gorbenko et al., 2020). On the other hand, replication means the same data is copied and kept on multiple connected machines. Given the trade-offs between consistency and availability, replication provides redundancy and increase data availability with the cost for reduced consistency or increased latency.

MongoDB offers configurable consistency levels to adapt different conditions and read/write concerns. Replication in MongoDB is asynchronous. In a replica set, there is one primary node and one or more secondary nodes. The primary node receives all writes and records every change in an operation log (oplog). The secondary nodes are copies of the primary node, which does not accept writes and replicate data by reading the oplog. In case of primary node failure, the secondary node is voted to be the new primary node (MongoDB, 2015). Data propagation occurs in the background, which allows quick response even when replicas are not fully synchronised (Ciurac et al., 2026).

CouchDB has strong local consistency with eventual cross-node consistency. To manage concurrent access to the database within a node, CouchDB implements Multi-Version Concurrency Control (MVCC). Each read request sees the latest snapshot at the start of the request, regardless of any changes that happen during the read. In term of distributed consistency, CouchDB implements eventual consistency. Each node or server sync periodically and the data is copied incrementally. When conflicts happened, a winning version is picked and the losing version would be kept in history, with the document being marked as conflicted and is managed later (Calçada & Bernardino, 2019). Replication in CouchDB is master-master or multi-master replication and its configurable. There are two types of replications, namely transient replication that is temporary and triggered manually, and persistent replication that is stored as a document and is automatically managed. Replication updates can be done through continuous replication or pull/push-based replication.

Firebase provides strong consistency and implements synchronous replication. Data is split into chunks, known as “splits”. Every split is replicated to different servers, with one of the replicas is selected as the leader. The leader-like coordinator is responsible for handling all write operations. When there is a write request, the leader proposes the change to other replicas, a majority of replicas must agree on the change to allow the write operation to be committed and all replicas apply the change. As all replicas are in sync with the latest committed data and user can read from the replica with strong consistency (Firebase, n.d.).

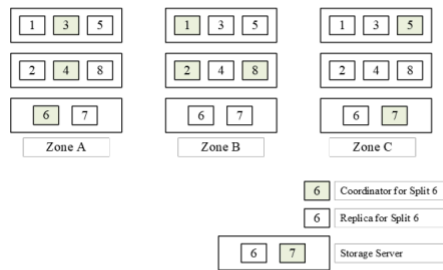


Fig. 1. Cloud Firestore Database Split (Firebase, n.d.).

4) **Scalability:** There are two types of scaling, namely vertical scaling and horizontal scaling. Vertical scaling means to increase the processing power of a single server or cluster. For databases hosted on premise, the hardware of the infrastructure has to be upgraded. For databases on cloud, which can be implemented in MongoDB Atlas and Couchbase Capella, vertical scaling can be upgraded with subscription tier. Firestore on the other hand handles scaling automatically, where it charges per usage. Horizontal scaling brings on additional nodes to share the load which can be done by partitioning. Data is partitioned into data chunks which are distributed across machines and replicated for reliability.

In MongoDB, horizontal partitioning or also known as sharding. Data is split into shards and each shard is implemented as a replica set. Each replica set is then replicated to multiple nodes. In MongoDB, the developers define the shard key to control the partitioning strategy (MongoDB, 2015).

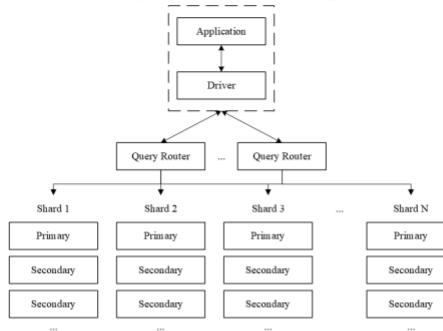


Fig. 2. Architecture of MongoDB in sharding (MongoDB, 2015)

CouchDB also partition data into shards and distribute copies of shards to different nodes in a cluster. In CouchDB, the databases are automatically shard and distributed, but the sharding behaviour can be configured.

Firebase splits data into “splits” and stored across distributed infrastructure. In Firebase, the partitioning is automatic and is handled fully by the system (Firebase, n.d.).

5) **Use case:** MongoDB is typically a general-purpose document database, with high flexibility and high read and write performance. Thus, MongoDB is well-suited for a wide range

of applications, including content management systems, flexible schema application and real-time analytics.

CouchDB on the other hand is optimised for offline-first apps and distributed systems due to its built-in replication mechanism and eventual synchronization.

Firestore is optimised for real-time apps due to its automatic data synchronisation and real-time update propagation. Hence, Firestore is most suited for mobile and web apps, which required real-time sync (Firestore, n.d.).

TABLE I. COMPARISON TABLE FOR MONGODB, COUCHDB AND FIRESTORE

Aspects	Databases		
	MongoDB	CouchDB	Firestore
Data model	BSON	JSON	JSON with hierarchical data
Query Mechanism	Rich query language	Mango queries, MapReduce views, HTTP/REST API access	Structured query API
Consistency	Eventual consistency, configurable	Strong consistency within a node, eventual consistency across nodes	Strong consistency
Replication	Asynchronous replication, Leader-Based Replication	Asynchronous replication, master-master / multi-master replication, configurable	Synchronous replication, Leader-Based Replication
Scalability	Sharding, controlled by user	Sharding, automatic but configurable	Partitioning, fully automatic
Use Cases	General-purpose apps, big data, flexible schema application	Offline-first apps, distributed systems	Mobile/web apps, real-time collaborative applications

C. Discussion

Within document-oriented databases, MongoDB and CouchDB share similarities in schema design and data models. However, MongoDB has a richer query capability, making it more suitable for applications that require complex data retrieval and analytics. While CouchDB emphasises offline-first capabilities through its replication model.

In terms of consistency, MongoDB provides configurable consistency levels, allowing developers to adjust according to different needs. In contrast with the limited consistency configurability in CouchDB and Firestore compared to MongoDB.

In terms of scalability, MongoDB supports horizontal scaling through user-defined sharding. This contrast with Firestore, which is fully managed.

Overall, MongoDB is a general-purpose NoSQL database that balances flexibility, scalability and query capability. While it may not be able to achieve the offline properties in CouchDB or real-time sync in Firestore, provides a versatile solution suitable for many modern applications.

CASE STUDY

An Integrated Note Learning System is built upon MongoDB according to Kate et al. (2026). It serves as a practical example of a data-intensive platform requiring efficient handling of structured and unstructured data. The system integrates notes, quizzes, multimedia content and semantic search.

MongoDB is adopted due to its flexible schema and its ability to manage different data with different characteristics through collections. The Atlas Vector Search enables semantic retrieval using embeddings, which allow users to perform semantic search across the learning materials. This capability is particularly significant for adaptive learning environments where traditional keyword-based retrieval is insufficient.

Compared to CouchDB and Firestore, MongoDB provides a stronger support for complex querying and semantic search integration. While CouchDB focuses more on replication and synchronisation and Firestore emphasises on real-time data, neither provides native support for vector-based retrieval at the same level. This makes MongoDB more suitable for systems requiring advanced data processing and retrieval.

However, there are also challenges in the implementation including the increase of complexity and computational overhead due to the AI-based embedding models.

CONCLUSION

In conclusion, MongoDB occupies a central position within the NoSQL ecosystem, particularly among document-oriented databases. Compared to CouchDB, MongoDB provides more advanced querying and analytics feature. When compared with Firestore, MongoDB offers greater control over deployment and data distribution. Overall, this analysis shows that MongoDB is a strong general-purpose NoSQL database.

REFERENCES

- Calçada, A., & Bernardino, J. (2019). *Evaluation of Couchbase, CouchDB and MongoDB using OSSpal*. 11th International Joint Conference on Knowledge Discovery, Knowledge Engineering and Knowledge Management (IC3K 2019). <https://doi.org/10.5220/0008345104270433>
- Ciumac, M., Györfi, C. A., Györfi, R. S., & Costea, F. M. (2026). Performance evaluation of MongoDB and RavenDB in IIOT-Inspired Data-Intensive mobile and web applications. *Future Internet*, 18(1), 57. <https://doi.org/10.3390/fi18010057>
- Firestore. (n.d.). *Understand reads and writes at scale*. <https://firebase.google.com/docs/firestore/understand-reads-writes-scale>
- Gorbenko, A., Karpenko, A., & Tarasyuk, O. (2020). *Analysis of Trade-offs in Fault-Tolerant Distributed Computing and Replicated Databases* (pp. 1–6). 11th IEEE International Conference on Dependable Systems, Services and Technologies, DESSERT'2020. <https://doi.org/10.1109/dessert50317.2020.9125078>
- Kate, P., Dagadkar, S., & Sontakke, A. (2026). Smart Note Management Learning Hub (pp. 1848–1853). 2026 7th International Conference on Mobile Computing and Sustainable Informatics (ICMCSI). <https://doi.org/10.1109/icmcsi67283.2026.11412869>
- Kesavan, R., Gay, D., Thevessen, D., Shah, J., & Mohan, C. (2023). *Firestore: The NoSQL Serverless Database for the Application Developer* (pp. 3376–3388). International Conference on Data Engineering. <https://doi.org/10.1109/icde55515.2023.00259>
- Matthew, H. (2015). A Case Study for NoSQL Applications and Performance Benefits: CouchDB vs. Postgres.
- MongoDB. (2015). *MongoDB Architecture Guide: MongoDB 3.2* [White paper].
- Truica, C., Radulescu, F., Boicea, A., & Bucur, I. (2015). *Performance Evaluation for CRUD Operations in Asynchronously Replicated Document Oriented Database* (pp. 191–196). 2015 20th International Conference on Control Systems and Computer Science, CSCS 2015. <https://doi.org/10.1109/cscs.2015.32>

Assignment 1 Lam Yoke Yu.pdf

ORIGINALITY REPORT

7 %

SIMILARITY INDEX

4 %

INTERNET SOURCES

1 %

PUBLICATIONS

5 %

STUDENT PAPERS

MATCH ALL SOURCES (ONLY SELECTED SOURCE PRINTED)

3%

★ medium.com

Internet Source

Exclude quotes On

Exclude bibliography On

Exclude matches Off